

Covid Classification using Pretrained ViT from tf_hub

```
In [1]: import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import cv2
import os
from tensorflow.keras.preprocessing import image
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from PIL import Image
```

Creating our Dataset

```
In [2]: from google.colab import drive
drive.mount('/content/gdrive')
```

Drive already mounted at /content/gdrive; to attempt to forcibly remount, call drive.mount("/content/gdrive", force_remount=True).

```
In [3]: image_directory = "/content/gdrive/MyDrive/Datasets/Covid-SARS-COV-2 Ct-Scan Dataset"
SIZE = 224
dataset = []
label = []
```

```
In [4]: #All images are of type .png

# Label 0 for Non-Covid images
non_covid_images = os.listdir(image_directory+'non-COVID/')
# Label 1 for Covid images
covid_images = os.listdir(image_directory+'COVID/')
```

```
In [5]: for i, image_name in enumerate(non_covid_images):
    if (image_name.split('.')[1]=='png'):
        image = cv2.imread(image_directory + 'non-COVID/' + image_name)
        image = Image.fromarray(image, 'RGB')
        image = image.resize((SIZE,SIZE))
        dataset.append(np.array(image))
        label.append(0)
```

```
In [6]: for i, image_name in enumerate(covid_images):
    if (image_name.split('.')[1]=='png'):
        image = cv2.imread(image_directory + 'COVID/' + image_name)
        image = Image.fromarray(image, 'RGB')
        image = image.resize((SIZE,SIZE))
        dataset.append(np.array(image))
        label.append(1)
```

```
In [7]: dataset = np.array(dataset)
label = np.array(label)
```

Train Test Split

```
In [8]: from sklearn.model_selection import train_test_split
```

```
In [9]: X_train, X_test, y_train, y_test = train_test_split(dataset, label, test_size=0.2,
```

```
In [10]: X_train.shape, X_test.shape, y_train.shape, y_test.shape
```

```
Out[10]: ((1984, 224, 224, 3), (497, 224, 224, 3), (1984,), (497,))
```

Scaling/Normalization

```
In [11]: from keras.utils import normalize
```

```
In [12]: X_train = normalize(X_train, axis=1)
X_test = normalize(X_test, axis=1)
```

Data Augmentation Layer

```
In [13]: data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    #layers.RandomRotation(0.2),
    #layers.RandomZoom(
    #    height_factor=0.2, width_factor=0.2
    #)
])
```

Pretrained-ViT

```
In [14]: import tensorflow_hub as hub
```

```
In [25]: model = tf.keras.Sequential([
    layers.InputLayer((224,224,3)),
    #Add Augmentation Layer
    data_augmentation,
    hub.KerasLayer("https://tfhub.dev/sayakpaul/vit_s16_fe/1", trainable=False),
    tf.keras.layers.Flatten(),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(512, activation='relu'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(256, activation='relu'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(32, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])
```

```
In [26]: model.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
sequential (Sequential)	(None, 224, 224, 3)	0
keras_layer_1 (KerasLayer)	(None, 384)	21665664
flatten_1 (Flatten)	(None, 384)	0
batch_normalization_3 (Batch Normalization)	(None, 384)	1536
dense_5 (Dense)	(None, 512)	197120
batch_normalization_4 (Batch Normalization)	(None, 512)	2048
dense_6 (Dense)	(None, 256)	131328
batch_normalization_5 (Batch Normalization)	(None, 256)	1024
dense_7 (Dense)	(None, 128)	32896
dense_8 (Dense)	(None, 64)	8256
dense_9 (Dense)	(None, 32)	2080
dense_10 (Dense)	(None, 1)	33
Total params: 22,041,985		
Trainable params: 374,017		
Non-trainable params: 21,667,968		

```
In [27]: model.compile(
            optimizer='adam',
            loss='binary_crossentropy',
            metrics=['accuracy']
        )
```

```
In [28]: history = model.fit(
            #training_set,
            x=X_train,
            y=y_train,
            #batch_size=batch_size,
            epochs=50,
        )
```

Epoch 1/50
62/62 [=====] - 21s 121ms/step - loss: 0.4305 - accuracy: 0.7959

Epoch 2/50
62/62 [=====] - 8s 124ms/step - loss: 0.2404 - accuracy: 0.8926

Epoch 3/50
62/62 [=====] - 8s 124ms/step - loss: 0.2176 - accuracy: 0.9123

Epoch 4/50
62/62 [=====] - 8s 126ms/step - loss: 0.1603 - accuracy: 0.9360

Epoch 5/50
62/62 [=====] - 8s 127ms/step - loss: 0.1167 - accuracy: 0.9536

Epoch 6/50
62/62 [=====] - 8s 128ms/step - loss: 0.1339 - accuracy: 0.9451

Epoch 7/50
62/62 [=====] - 8s 132ms/step - loss: 0.1013 - accuracy: 0.9637

Epoch 8/50
62/62 [=====] - 8s 132ms/step - loss: 0.0800 - accuracy: 0.9662

Epoch 9/50
62/62 [=====] - 8s 134ms/step - loss: 0.0870 - accuracy: 0.9682

Epoch 10/50
62/62 [=====] - 8s 132ms/step - loss: 0.0885 - accuracy: 0.9688

Epoch 11/50
62/62 [=====] - 8s 130ms/step - loss: 0.0545 - accuracy: 0.9829

Epoch 12/50
62/62 [=====] - 8s 131ms/step - loss: 0.0816 - accuracy: 0.9682

Epoch 13/50
62/62 [=====] - 8s 128ms/step - loss: 0.0516 - accuracy: 0.9814

Epoch 14/50
62/62 [=====] - 8s 129ms/step - loss: 0.0760 - accuracy: 0.9708

Epoch 15/50
62/62 [=====] - 8s 128ms/step - loss: 0.0466 - accuracy: 0.9829

Epoch 16/50
62/62 [=====] - 8s 129ms/step - loss: 0.0673 - accuracy: 0.9763

Epoch 17/50
62/62 [=====] - 8s 130ms/step - loss: 0.0772 - accuracy: 0.9728

Epoch 18/50
62/62 [=====] - 8s 129ms/step - loss: 0.0299 - accuracy: 0.9894

Epoch 19/50
62/62 [=====] - 8s 131ms/step - loss: 0.0465 - accuracy: 0.9814

Epoch 20/50
62/62 [=====] - 8s 130ms/step - loss: 0.0533 - accuracy: 0.9798

Epoch 21/50
62/62 [=====] - 8s 131ms/step - loss: 0.0140 - accuracy: 0.9965

Epoch 22/50

```
62/62 [=====] - 8s 131ms/step - loss: 0.0532 - accuracy:
0.9824
Epoch 23/50
62/62 [=====] - 8s 130ms/step - loss: 0.0417 - accuracy:
0.9859
Epoch 24/50
62/62 [=====] - 8s 131ms/step - loss: 0.0583 - accuracy:
0.9803
Epoch 25/50
62/62 [=====] - 8s 129ms/step - loss: 0.0461 - accuracy:
0.9849
Epoch 26/50
62/62 [=====] - 8s 130ms/step - loss: 0.0437 - accuracy:
0.9844
Epoch 27/50
62/62 [=====] - 8s 129ms/step - loss: 0.0282 - accuracy:
0.9879
Epoch 28/50
62/62 [=====] - 8s 129ms/step - loss: 0.0381 - accuracy:
0.9869
Epoch 29/50
62/62 [=====] - 8s 129ms/step - loss: 0.0270 - accuracy:
0.9914
Epoch 30/50
62/62 [=====] - 8s 128ms/step - loss: 0.0296 - accuracy:
0.9874
Epoch 31/50
62/62 [=====] - 8s 130ms/step - loss: 0.0545 - accuracy:
0.9829
Epoch 32/50
62/62 [=====] - 8s 128ms/step - loss: 0.0354 - accuracy:
0.9884
Epoch 33/50
62/62 [=====] - 8s 130ms/step - loss: 0.0424 - accuracy:
0.9844
Epoch 34/50
62/62 [=====] - 8s 129ms/step - loss: 0.0278 - accuracy:
0.9904
Epoch 35/50
62/62 [=====] - 8s 129ms/step - loss: 0.0234 - accuracy:
0.9919
Epoch 36/50
62/62 [=====] - 8s 130ms/step - loss: 0.0186 - accuracy:
0.9929
Epoch 37/50
62/62 [=====] - 8s 128ms/step - loss: 0.0489 - accuracy:
0.9859
Epoch 38/50
62/62 [=====] - 8s 130ms/step - loss: 0.0228 - accuracy:
0.9909
Epoch 39/50
62/62 [=====] - 8s 130ms/step - loss: 0.0165 - accuracy:
0.9950
Epoch 40/50
62/62 [=====] - 8s 130ms/step - loss: 0.0301 - accuracy:
0.9914
Epoch 41/50
62/62 [=====] - 8s 133ms/step - loss: 0.0225 - accuracy:
0.9909
Epoch 42/50
62/62 [=====] - 8s 129ms/step - loss: 0.0250 - accuracy:
0.9914
Epoch 43/50
62/62 [=====] - 8s 130ms/step - loss: 0.0190 - accuracy:
```

```

0.9929
Epoch 44/50
62/62 [=====] - 8s 129ms/step - loss: 0.0204 - accuracy:
0.9934
Epoch 45/50
62/62 [=====] - 8s 130ms/step - loss: 0.0217 - accuracy:
0.9919
Epoch 46/50
62/62 [=====] - 8s 130ms/step - loss: 0.0400 - accuracy:
0.9869
Epoch 47/50
62/62 [=====] - 8s 129ms/step - loss: 0.0218 - accuracy:
0.9934
Epoch 48/50
62/62 [=====] - 8s 130ms/step - loss: 0.0330 - accuracy:
0.9899
Epoch 49/50
62/62 [=====] - 8s 129ms/step - loss: 0.0267 - accuracy:
0.9919
Epoch 50/50
62/62 [=====] - 8s 130ms/step - loss: 0.0106 - accuracy:
0.9970

```

Performance Evaluation

In [29]: `model.evaluate(X_test,y_test)`

```

16/16 [=====] - 12s 457ms/step - loss: 0.1301 - accuracy:
0.9638

```

Out[29]: `[0.1300784796476364, 0.9637826681137085]`

In [30]: `from sklearn.metrics import accuracy_score,classification_report,confusion_matrix`

In [31]: `threshold = 0.5`
`y_pred = (model.predict(X_test)>=threshold).astype(int)`

```

16/16 [=====] - 3s 126ms/step

```

In [32]: `y_pred.shape, y_test.shape`

Out[32]: `((497, 1), (497,))`

In [33]: `cm=confusion_matrix(y_test, y_pred)`
`print(cm)`

```

[[228  10]
 [  8 251]]

```

In [34]: `print(classification_report(y_test,y_pred))`

	precision	recall	f1-score	support
0	0.97	0.96	0.96	238
1	0.96	0.97	0.97	259
accuracy			0.96	497
macro avg	0.96	0.96	0.96	497
weighted avg	0.96	0.96	0.96	497

In []:

